

OT Attack & Detection Lab

OpenPLC | Modbus | Kali Linux

Simulating, Attacking, and Defending
Industrial Control Systems

Project Type:	Cybersecurity Research Lab
Environment:	VMware Workstation (Ubuntu 24.04 + Kali Linux)
Target System:	OpenPLC Runtime — Water Tank Controller
Attack Vector:	Modbus TCP Write (Function Code 6)
Detection Stack:	Wireshark + Suricata IDS + Wazuh SIEM

Abdul Moaid

IT Lead & GRC Consultant | Kinverg (Pvt.) Ltd.
Associate of (ISC)² | BS Cybersecurity, Air University

July 2026
Lahore, Pakistan

Table of Contents

1. Executive Summary	3
2. Lab Environment and Network Architecture	4
3. PLC Program Design	5
4. Attack Implementation	7
5. Wireshark Traffic Analysis	8
6. Suricata IDS Detection	9
7. Wazuh SIEM Integration	10
8. MITRE ATT&CK; Mapping	11
9. Key Findings and Recommendations	12

1. Executive Summary

This report documents the design, execution, and detection of a simulated cyberattack against an Operational Technology (OT) environment. The target was a water treatment plant controller running on OpenPLC Runtime. The attacker machine was Kali Linux. The entire lab ran inside VMware Workstation on a private virtual network.

The attack used Modbus TCP protocol to send unauthorized write commands to the PLC. These commands manipulated the water tank sensor values directly. The PLC had no authentication on its Modbus interface. This is a realistic scenario because most legacy industrial systems still operate without any form of protocol level security.

Three detection layers were deployed to catch the attack. Wireshark captured the raw network traffic and showed every Modbus packet in detail. Suricata IDS triggered custom rules that flagged unauthorized Function Code 6 (Write Single Register) traffic. Wazuh SIEM ingested the Suricata alerts and correlated them into a centralized dashboard with severity classification.

The attack was successful. Register values changed from [0, 0, 0] to [1, 0, 1]. The pump turned on. The alarm activated. Every detection tool caught it. This proves that even a basic Modbus write from an unauthorized host can cause physical process disruption. It also proves that open source tools can detect these attacks effectively when configured correctly.

2. Lab Environment and Network Architecture

The lab was built on VMware Workstation using two virtual machines connected through a host only network. No traffic left the physical host. This isolated setup prevented any accidental exposure to production networks.

Network Configuration

Component	Operating System	IP Address	Role
PLC Server	Ubuntu 24.04 LTS	192.168.35.128	OpenPLC Runtime, Suricata, Wazuh Agent
Attacker	Kali Linux (Rolling)	192.168.35.129	Nmap Scanning, Modbus Attack Script
SIEM Server	Ubuntu (same host)	192.168.71.128	Wazuh Manager and Dashboard

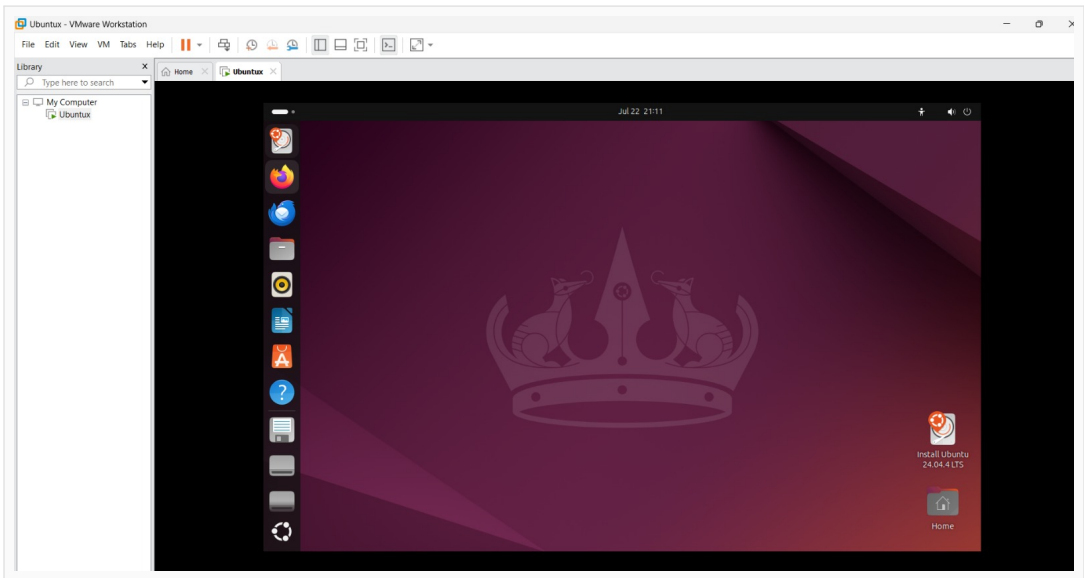


Figure 1: Ubuntu VM running inside VMware Workstation. This machine hosted the OpenPLC Runtime.

The PLC server listened on TCP port 502 for Modbus connections and TCP port 8080 for the OpenPLC web interface. Suricata monitored the ens33 network interface in passive mode. Wazuh agent forwarded all Suricata alerts to the Wazuh manager for centralized analysis.

3. PLC Program Design

The PLC program simulated a basic water tank control system. It was written in Ladder Diagram (LD) language and uploaded through the OpenPLC web editor. The program was named "water_tank" and described as "Used in controlling the water tank."

Register Mapping

The program used three Modbus holding registers (MW0, MW1, MW2) and two digital outputs (QX0.0, QX0.1). Each register represented a specific process variable.

Point Name	Type	Location	Modbus Address	Purpose
low_level	WORD	%MW0	0	Water low level sensor
high_level	WORD	%MW1	1	Water high level sensor
alarm_level	WORD	%MW2	2	Alarm trigger threshold
pump	BOOL	%QX0.0	Coil 0	Pump motor output
alarm	BOOL	%QX0.1	Coil 1	Alarm buzzer output

Ladder Logic

The ladder diagram had two rungs. Rung 1 checked if low_level was TRUE and high_level was FALSE (normally closed contact). When both conditions were met, the pump output energized. Rung 2 checked the alarm_level register. If it was TRUE, the alarm coil activated. This logic meant that writing a 1 to MW0 and MW2 from an external source would force the pump and alarm on without any physical sensor input.

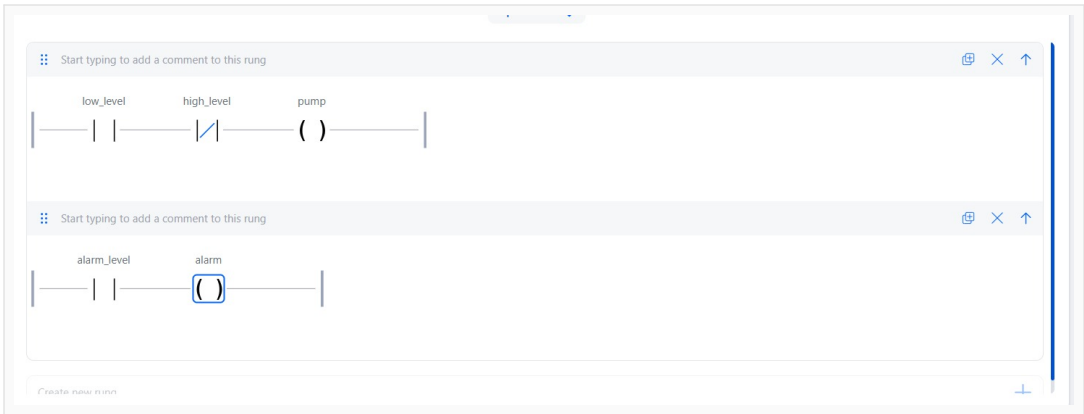


Figure 2: Ladder Diagram showing two rungs. Rung 1 controls the pump. Rung 2 controls the alarm.

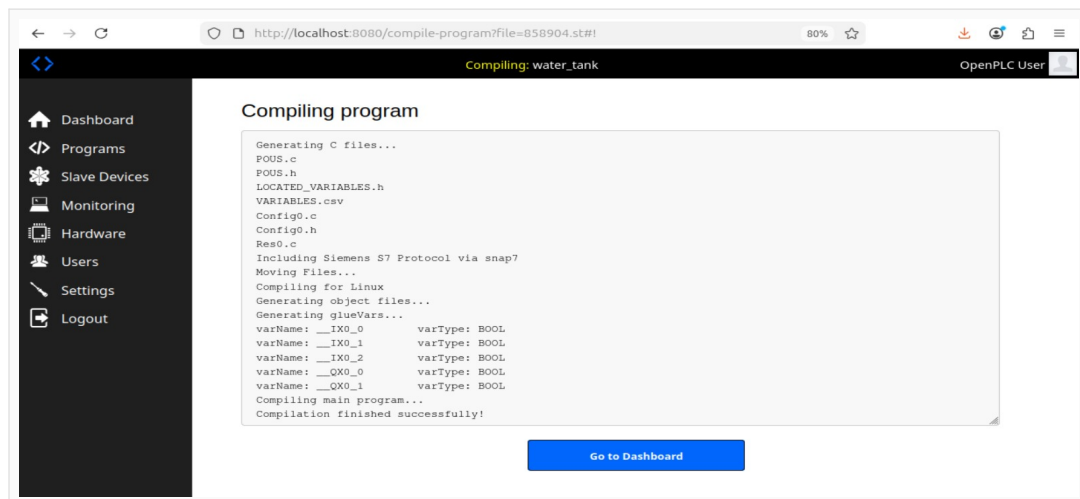


Figure 3: OpenPLC compilation output. The program compiled successfully with all variables mapped.

The `LOCATED_VARIABLES.h` file confirmed the register assignments. `MW0` mapped to address 0, `MW1` to address 1, and `MW2` to address 2. This file was critical for verifying that the attack script targeted the correct Modbus addresses.

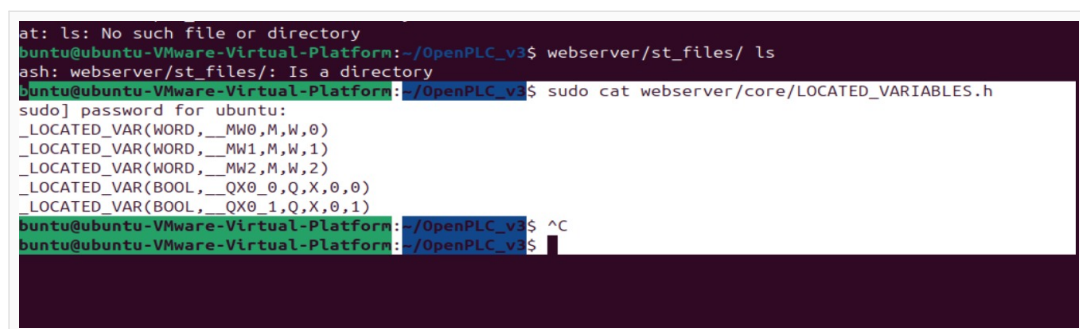


Figure 4: `LOCATED_VARIABLES.h` showing the exact register to address mapping used by OpenPLC.

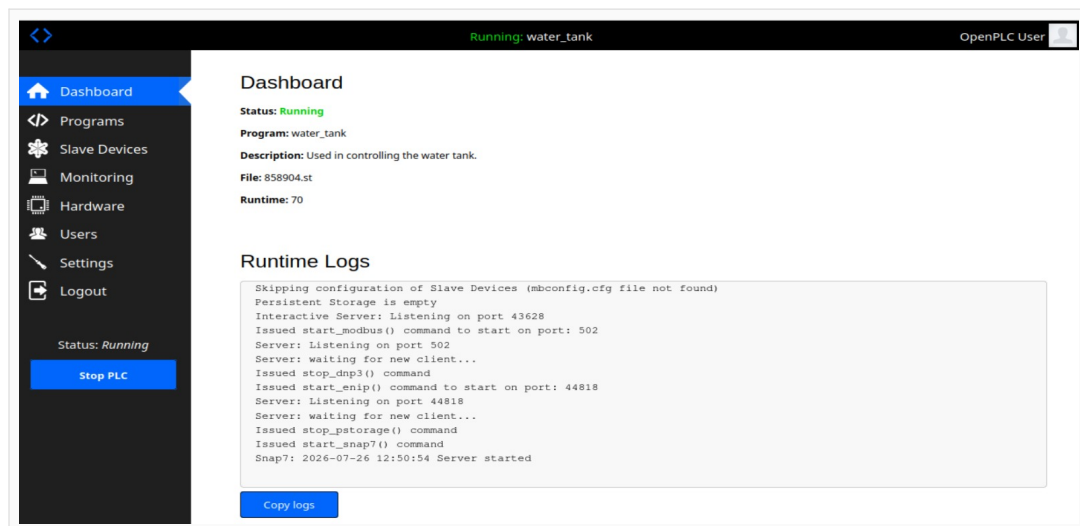


Figure 5: OpenPLC Dashboard showing the `water_tank` program in Running state with Modbus server on port 502.

The runtime logs confirmed that the Modbus server started on port 502 and was waiting for client connections. The Snap7 server also started for Siemens S7 protocol support. At this point the PLC was live and accepting connections from any host on the network.

4. Attack Implementation

The attack was executed from the Kali Linux machine using a Python script built with the pymodbus library. The script connected to the OpenPLC Modbus server on port 502 and performed two operations. First it read the current register values. Then it wrote new values to manipulate the process.

Pre Attack State

Before running the attack, the OpenPLC monitoring page showed all registers at zero. The pump indicator was red (FALSE). The alarm indicator was also red (FALSE). The system was in a safe idle state with no active process.

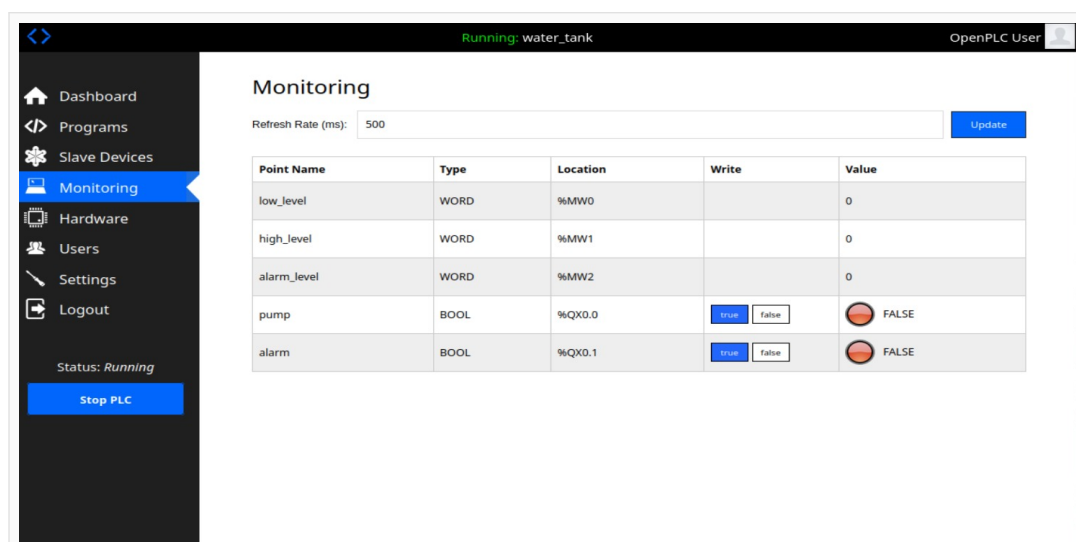
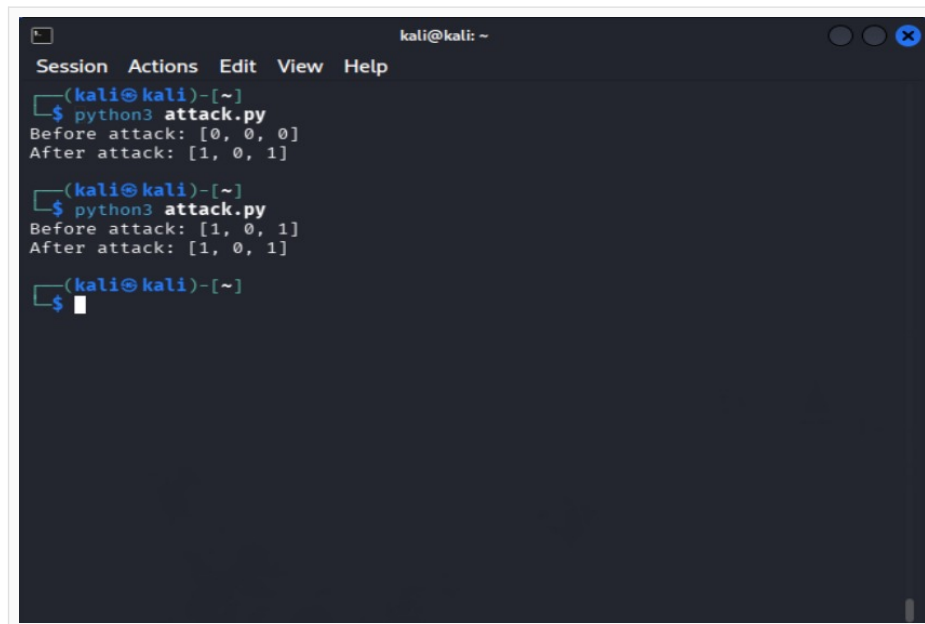


Figure 6: OpenPLC Monitoring page before the attack. All values are 0. Pump and alarm are OFF.

Attack Execution

The Python attack script wrote value 1 to register address 0 (low_level) and value 1 to register address 2 (alarm_level). The script confirmed the change by reading registers again immediately after writing. The output showed the values changed from [0, 0, 0] to [1, 0, 1].

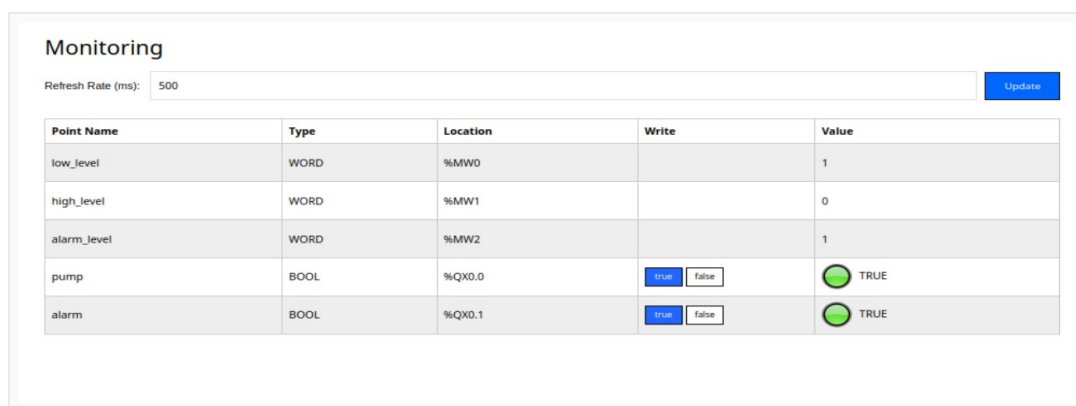


```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ python3 attack.py  
Before attack: [0, 0, 0]  
After attack: [1, 0, 1]  
  
(kali@kali)-[~]  
$ python3 attack.py  
Before attack: [1, 0, 1]  
After attack: [1, 0, 1]  
  
(kali@kali)-[~]  
$
```

Figure 7: Kali terminal showing `attack.py` execution. Registers changed from `[0, 0, 0]` to `[1, 0, 1]`.

Post Attack State

After the attack, the OpenPLC monitoring page reflected the tampered values. The `low_level` register showed 1. The `alarm_level` register showed 1. The pump indicator turned green (TRUE). The alarm indicator also turned green (TRUE). The attacker had successfully manipulated the physical process outputs from a remote machine without any credentials or authentication.



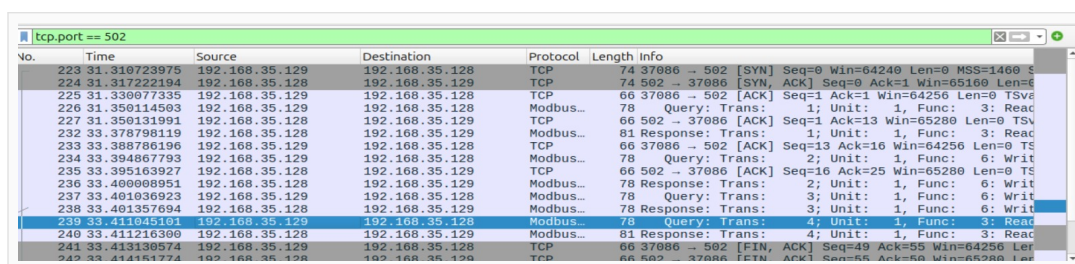
Point Name	Type	Location	Write	Value
low_level	WORD	%MW0		1
high_level	WORD	%MW1		0
alarm_level	WORD	%MW2		1
pump	BOOL	%QX0.0	☒ true ☐ false	 TRUE
alarm	BOOL	%QX0.1	☒ true ☐ false	 TRUE

Figure 8: OpenPLC Monitoring after the attack. Pump and alarm are now TRUE. Values manipulated remotely.

5. Wireshark Traffic Analysis

Wireshark captured all network traffic during the attack. The display filter "tcp.port == 502" was applied to isolate Modbus traffic. The capture showed the complete attack sequence from TCP handshake to the final Modbus read.

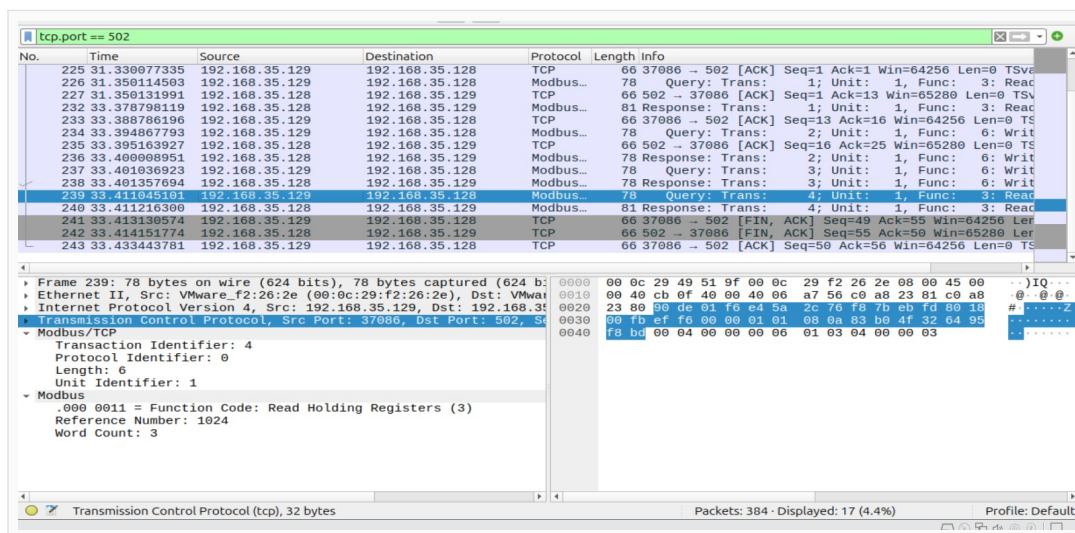
The traffic pattern was clear. Packets 223 through 225 showed the TCP three way handshake (SYN, SYN ACK, ACK) from 192.168.35.129 to 192.168.35.128. This was the attacker establishing a connection. The next packets contained Modbus queries with Function Code 3 (Read Holding Registers) and Function Code 6 (Write Single Register). The write commands were the actual attack payloads.



No.	Time	Source	Destination	Protocol	Length	Info
223	31.310723975	192.168.35.129	192.168.35.128	TCP	74	37086 → 502 [SYN] Seq=0 Win=1460 Len=0
224	31.317222194	192.168.35.128	192.168.35.129	TCP	74	502 → 37086 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0
225	31.330077335	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [ACK] Seq=1 Ack=13 Win=64256 Len=0
226	31.350114503	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 1; Unit: 1, Func: 3: Read
227	31.350131991	192.168.35.128	192.168.35.129	TCP	66	502 → 37086 [ACK] Seq=1 Ack=13 Win=65280 Len=0
232	33.378798119	192.168.35.128	192.168.35.129	Modbus...	81	Response: Trans: 1; Unit: 1, Func: 3: Read
233	33.388786196	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [ACK] Seq=13 Ack=16 Win=64256 Len=0
234	33.394867793	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 2; Unit: 1, Func: 6: Write
235	33.395163927	192.168.35.128	192.168.35.129	TCP	66	502 → 37086 [ACK] Seq=16 Ack=25 Win=65280 Len=0
236	33.400008951	192.168.35.128	192.168.35.129	Modbus...	78	Response: Trans: 2; Unit: 1, Func: 6: Write
237	33.401036923	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 3; Unit: 1, Func: 6: Write
238	33.401357694	192.168.35.128	192.168.35.129	Modbus...	78	Response: Trans: 3; Unit: 1, Func: 6: Write
239	33.411045101	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 4; Unit: 1, Func: 3: Read
240	33.411216300	192.168.35.128	192.168.35.129	Modbus...	81	Response: Trans: 4; Unit: 1, Func: 3: Read
241	33.413130574	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [FIN, ACK] Seq=55 Ack=55 Win=64256 Len=0
242	33.414151774	192.168.35.128	192.168.35.129	TCP	66	502 → 37086 [FIN, ACK] Seq=55 Ack=55 Win=65280 Len=0

Figure 9: Wireshark capture filtered on tcp.port == 502. Shows the full Modbus conversation between attacker and PLC.

Deep packet inspection of frame 239 revealed the Modbus protocol structure. The Transaction Identifier was 4. The Unit Identifier was 1. The Function Code was 3 (Read Holding Registers) with a Reference Number of 1024 and Word Count of 3. This was the attacker verifying the register values after the write operation.



No.	Time	Source	Destination	Protocol	Length	Info
225	31.330077335	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [ACK] Seq=1 Ack=13 Win=64256 Len=0
226	31.350114503	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 1; Unit: 1, Func: 3: Read
227	31.350131991	192.168.35.128	192.168.35.129	TCP	66	502 → 37086 [ACK] Seq=1 Ack=13 Win=65280 Len=0
232	33.378798119	192.168.35.128	192.168.35.129	Modbus...	81	Response: Trans: 1; Unit: 1, Func: 3: Read
233	33.388786196	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [ACK] Seq=13 Ack=16 Win=64256 Len=0
234	33.394867793	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 2; Unit: 1, Func: 6: Write
235	33.395163927	192.168.35.128	192.168.35.129	TCP	66	502 → 37086 [ACK] Seq=16 Ack=25 Win=65280 Len=0
236	33.400008951	192.168.35.128	192.168.35.129	Modbus...	78	Response: Trans: 2; Unit: 1, Func: 6: Write
237	33.401036923	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 3; Unit: 1, Func: 6: Write
238	33.401357694	192.168.35.128	192.168.35.129	Modbus...	78	Response: Trans: 3; Unit: 1, Func: 6: Write
239	33.411045101	192.168.35.129	192.168.35.128	Modbus...	78	Query: Trans: 4; Unit: 1, Func: 3: Read
240	33.411216300	192.168.35.128	192.168.35.129	Modbus...	81	Response: Trans: 4; Unit: 1, Func: 3: Read
241	33.413130574	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [FIN, ACK] Seq=55 Ack=55 Win=64256 Len=0
242	33.414151774	192.168.35.128	192.168.35.129	TCP	66	502 → 37086 [FIN, ACK] Seq=55 Ack=55 Win=65280 Len=0
243	33.433443781	192.168.35.129	192.168.35.128	TCP	66	37086 → 502 [ACK] Seq=58 Ack=56 Win=64256 Len=0

Frame 239: 78 bytes on wire (624 bits), 78 bytes captured (624 b)	0000	00 0c 29 49 51 9f 00 0c 29 f2 26 2e 08 00 45 00	..)IQ...
Ethernet II, Src: VMWare f2:26:2e (00:0c:29:f2:26:2e), Dst: VMWare	0010	00 40 cb 0f 40 00 40 06 a7 56 c0 a8 23 81 c0 a8	...@...@...
Internet Protocol Version 4, Src: 192.168.35.129, Dst: 192.168.35.128	0020	23 80 00 d6 81 f6 e4 5a 2c 76 f8 70 cb fd 89 18	...#.....2
Transmission Control Protocol, Src Port: 37086, Dst Port: 502, Seq=55, Ack=55, Win=64256, Len=0	0030	00 fb ef f6 00 00 01 01 08 0a 83 b0 4f 32 64 95	
Modbus/TCP	0040	f8 bd 00 04 00 00 00 06 01 03 04 00 00 03	
Transaction Identifier: 4			
Protocol Identifier: 0			
Length: 6			
Unit Identifier: 1			
Modbus			
0000011 = Function Code: Read Holding Registers (3)			
Reference Number: 1024			
Word Count: 3			

Figure 10: Wireshark deep packet inspection showing Modbus PDU fields including Function Code, Reference Number, and raw hex bytes.

6. Suricata IDS Detection

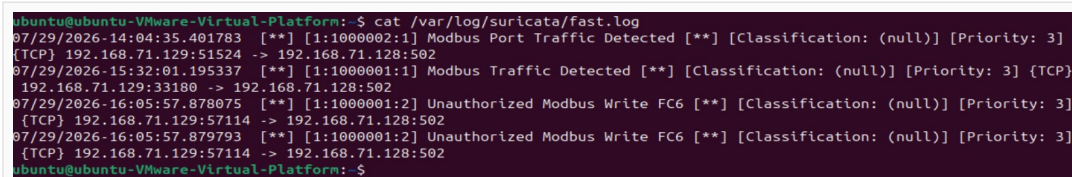
Suricata was deployed on the PLC server machine in IDS mode. Two custom rules were written to detect Modbus traffic. Rule 1000002 detected any Modbus port traffic (general awareness). Rule 1000001 specifically detected unauthorized Modbus Write commands using Function Code 6.

Custom Suricata Rules

The first rule triggered on any TCP traffic to port 502 with the signature "Modbus Port Traffic Detected" at Priority 3. The second rule matched Modbus Write Single Register (FC6) by inspecting the byte at offset 7 in the TCP payload for the hex value 06. It fired with the signature "Unauthorized Modbus Write FC6" also at Priority 3.

Detection Results

The fast.log file recorded four alerts during the attack window. Two alerts were general Modbus traffic detections from the earlier reconnaissance phase. Two alerts were specific FC6 write detections that fired at 16:05:57 on July 29, 2026. Both FC6 alerts showed the attacker IP 192.168.71.129 targeting the PLC at 192.168.71.128 on port 502.



```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ cat /var/log/suricata/fast.log
07/29/2026-14:04:35.401783  [**] [1:1000002:1] Modbus Port Traffic Detected [**] [Classification: (null)] [Priority: 3]
{TCP} 192.168.71.129:51524 -> 192.168.71.128:502
07/29/2026-15:32:01.195337  [**] [1:1000001:1] Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3] {TCP}
192.168.71.129:33180 -> 192.168.71.128:502
07/29/2026-16:05:57.878075  [**] [1:1000001:2] Unauthorized Modbus Write FC6 [**] [Classification: (null)] [Priority: 3]
{TCP} 192.168.71.129:57114 -> 192.168.71.128:502
07/29/2026-16:05:57.879793  [**] [1:1000001:2] Unauthorized Modbus Write FC6 [**] [Classification: (null)] [Priority: 3]
{TCP} 192.168.71.129:57114 -> 192.168.71.128:502
ubuntu@ubuntu-VMware-Virtual-Platform:~$
```

Figure 11: Suricata fast.log showing four alerts. The last two are Unauthorized Modbus Write FC6 detections.

7. Wazuh SIEM Integration

Wazuh SIEM was the final layer of the detection stack. It ingested Suricata alerts through the Wazuh agent running on the PLC server. A custom decoder parsed the Suricata eve.json output. A custom rule (ID 100001, Level 13) correlated the Modbus write alerts into actionable security events visible on the Wazuh dashboard.

Dashboard Overview

The Wazuh dashboard displayed the overall security posture. In the last 24 hours there were 0 Critical severity alerts, 0 High severity alerts, 12 Medium severity alerts (Rule level 7 to 11), and 31 Low severity alerts (Rule level 0 to 6). The Modbus write alerts fell into the Medium category.

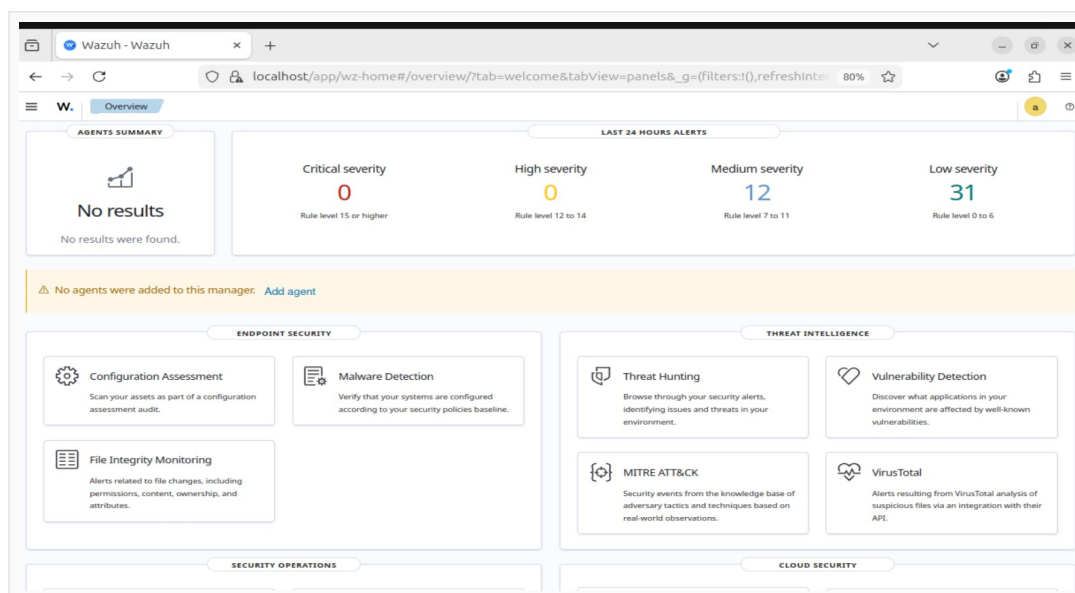


Figure 12: Wazuh Overview Dashboard showing alert severity distribution. 12 Medium and 31 Low severity events recorded.

Alert Details

Drilling into the alerts revealed the Suricata integration working correctly. Each alert entry showed the timestamp, agent name (ubuntu VMware Virtual Platform), source IP (192.168.71.129), source port, and the alert signature. The data fields included data.alert.signature showing 'Unauthorized Modbus Write FC6' and data.alert.severity at level 3.

Columns	Sort fields
Time (timestamp)	Source
Jul 29, 2026 @ 16:05:57.986	input.type log agent.name ubuntu-VMware-Virtual-Platform agent.id 000 manager.name ubuntu-VMware-Virtual-Platform data.capture_file /var/log/suricata/log.pcap.1785323143 data.app_proto modbus data.in_iface ens33 data.src_ip 192.168.71.129 data.src_port 57114 data.a.event_type alert data.alert.severity 3 data.alert.signature_id 1000001 data.alert.rev 2 data.alert.gid 1 data.alert.signature Unauthorized...
Jul 29, 2026 @ 16:05:57.986	input.type log agent.name ubuntu-VMware-Virtual-Platform agent.id 000 manager.name ubuntu-VMware-Virtual-Platform data.capture_file /var/log/suricata/log.pcap.1785323143 data.app_proto modbus data.in_iface ens33 data.src_ip 192.168.71.129 data.src_port 57114 data.a.event_type alert data.alert.severity 3 data.alert.signature_id 1000001 data.alert.rev 2 data.alert.gid 1 data.alert.signature Unauthorized...
Jul 29, 2026 @ 15:32:02.500	input.type log agent.name ubuntu-VMware-Virtual-Platform agent.id 000 manager.name ubuntu-VMware-Virtual-Platform data.in_iface ens33 data.src_ip 192.168.71.129 data.src_port 33180 data.event_type alert data.alert.severity 3 data.alert.signature_id 1000001 data.alert.rev 1 data.alert.gid 1 data.alert.signature Modbus Traffic Detected data.alert.action allowed data.flow_id 557495280189287.000000 dat...

Figure 13: Wazuh alert log showing three Suricata events ingested. Two FC6 write alerts and one general Modbus traffic alert.

Columns		Sort fields	
Time (timestamp)		Source	
Jul 29, 2026 @ 16:05:57.986	input.type	log	agent.name
	3	data.app.proto	modbus
Jul 29, 2026 @ 16:05:57.986	data.in_iface	ens33	data.src_ip
	192.168.71.129	data.src_port	57114
Jul 29, 2026 @ 16:05:57.986	data.event_type	alert	data.alert.severity
	3	data.alert.signature_id	1000001
Jul 29, 2026 @ 16:05:57.986	data.alert.rev	2	data.alert.gid
	1	data.alert.signature	Unauthorized Modbus Write FC6
Jul 29, 2026 @ 16:05:57.986	data.alert.action	allowed	data.modbus.request.transaction_id
	1	data.modbus.request.access_t...	

Figure 14: Expanded Wazuh alert view showing Modbus Write FC6 detections with transaction ID and flow data.

Custom Wazuh Rule

The custom rule was stored in `modbus_wazuh.xml` under `etc/rules`. It had Rule ID 100001, Level 13, and belonged to the Modbus and Suricata groups. The rule matched on the alert signature pattern "Unauthorized Modbus Write" from the Suricata decoder. The MITRE compliance tag was attached for framework alignment.

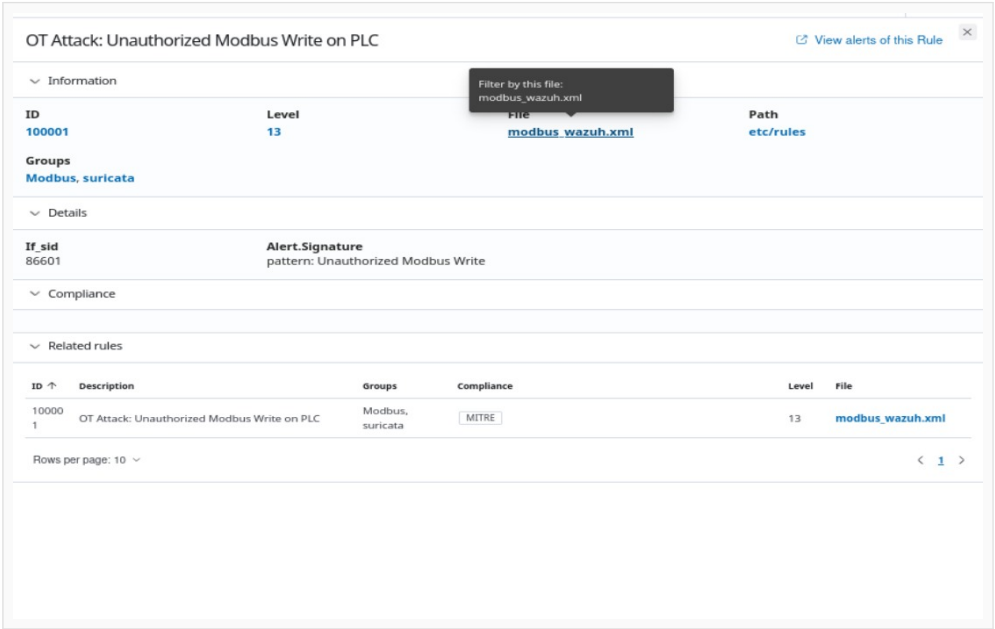


Figure 15: Wazuh Rule Manager showing custom Rule 100001 with Level 13, MITRE compliance, and `modbus_wazuh.xml` source file.

8. MITRE ATT&CK; Mapping

Every phase of the attack was mapped to the MITRE ATT&CK; for ICS framework. This mapping connects the lab activities to real world adversary behavior documented by MITRE. It also helps prioritize defensive controls by understanding which tactics an attacker used.

Tactic	Technique ID	Technique Name	Lab Evidence
Discovery	T0846	Remote System Discovery	Nmap scan identified port 502 open on the PLC
Lateral Movement	T0886	Remote Services	Modbus TCP connection from Kali to OpenPLC
Impair Process Control	T0855	Unauthorized Command Message	Write Single Register (FC6) commands sent
Inhibit Response Function	T0809	Data Destruction	Original sensor values overwritten
Impact	T0831	Manipulation of Control	Pump forced ON, alarm forced ON

9. Key Findings and Recommendations

Findings

F1: No Authentication on Modbus

The Modbus TCP protocol has zero built in authentication. Any device on the network can connect to port 502 and write arbitrary values to PLC registers. This is by design in the protocol specification and represents the single largest risk in legacy OT environments.

F2: Successful Process Manipulation

The attacker changed physical process outputs (pump and alarm) using only three lines of Python code. No credentials were needed. No exploit was required. The protocol itself was the vulnerability.

F3: Detection Was Effective

All three detection tools caught the attack. Wireshark provided forensic level detail. Suricata fired automated alerts within milliseconds. Wazuh aggregated everything into a single dashboard. The defense in depth approach worked.

F4: Alert Correlation Gap

While each tool detected the attack independently, there was no automated response. The alerts were informational only. No firewall rule was triggered. No connection was terminated. An attacker with sustained access could continue writing values.

Recommendations

R1: Network Segmentation

Place all PLC and SCADA devices on a dedicated VLAN. Use firewall rules to restrict Modbus traffic to authorized engineering workstations only. Block all other hosts from reaching port 502.

R2: Modbus Protocol Whitelisting

Configure Suricata or a dedicated OT firewall to whitelist only specific Modbus function codes from specific source IPs. Any FC6 write from an unknown source should be blocked and alerted on simultaneously.

R3: Active Response Integration

Connect Wazuh active response module to the host firewall (iptables). When Rule 100001 fires, automatically block the attacker IP at the network level. This converts passive detection into active prevention.

R4: Regular VAPT on OT Networks

Schedule periodic vulnerability assessments and penetration tests specifically for OT protocols. Test for unauthorized Modbus access, default credentials on HMI/SCADA, and unpatched firmware on PLCs.

End of Report